DeFuzz: Invariant-Guided Agentic Testing of Compiler-Instrumented Defenses

Jingyao Zhang
School of Cyber Science and Engineering
Southeast University
Nanjing, China
zhangjingyao@seu.edu.cn

Abstract— Compiler-instrumented defenses (e.g., stack canaries, Control-Flow Integrity (CFI), and `_FORTIFY_SOURCE`) protect software against security threats, such as memory corruption, control-flow hijacking, and data leakage. However, these defenses can fail silently: the compiler produces a working binary without any warnings or crashes, yet the security protection is quietly weakened or broken. For example, in CVE-2023-4039, dynamic stack allocation allowed an overflow to bypass the GCC stack canary on AArch64. Existing compiler testing tools fail to detect such bugs. Conventional crash and differential testing only check whether a program produces correct outputs, leaving a fundamental oracle gap when a defense is silently weakened or broken.

In this paper, we present DEFUZZ, an invariant-directed agentic framework for discovering silent failures in compiler-instrumented defenses. DEFUZZ first extracts security invariants from compiler source code, official compiler and ISA documentation, and historical bug fixes using a hybrid CoT and RAG pipeline. It then maps these invariants to compiler passes and decision branches, locating where defenses are implemented and controlled. To detect violations reliably, DEFUZZ formalizes these invariants in a predicate IR and lowers them into programmatic bug oracles, implementing them as static and dynamic checkers across target ISAs. All bug verdicts are decided strictly by these checkers, with zero LLM involvement in bug adjudication. Guided by the mapped compiler locations, specialized LLM agents perform directed fuzzing to hit targeted defense branches, synthesizing and mutating seeds to trigger invariant violations under oracle evaluation. We evaluated DEFUZZ on GCC and LLVM across multiple ISAs. DEFUZZ discovered 126 previously unknown silent failures in compiler-instrumented defenses, of which 34 have been confirmed upstream, including 11 fixed. Some of these bugs were rated as high-severity vulnerabilities. These results show that DEFUZZ effectively uncovers critical silent defense bugs that evade existing compiler testing techniques.

I. INTRODUCTION

Compilers deploy a wide range of compiler-instrumented defenses to harden software against security threats, such as memory corruption, control-flow hijacking, and sensitive data leakage. During compilation, these defenses insert runtime bounds checks, enforce stack layout rules, or clear registers to block exploit primitives. However, implementing these defenses accurately inside production compilers like GCC and LLVM is difficult. Defense code must interact with complex optimization passes and translate abstract protection rules into machine instructions across diverse target ISAs. This translation relies on strict assumptions about object bounds, instruction semantics, register conventions, and stack frame layouts. When the implementation of a defense mechanism fails to match these ISA or optimization assumptions, its security protection breaks from within.

When a defense implementation goes wrong, it often causes **silent defense failures**: the compiler produces a working binary without any warnings or crashes, yet the security protection is quietly weakened or broken. Because the compiled program still runs normally on benign inputs, standard functional tests pass completely, leaving the software exposed to attack. For example, in **CVE-2023-4039** [1], GCC on AArch64 placed the saved frame pointer and return address at the bottom of the fixed stack frame, below the stack canary. When a function used `alloca` or a variable-length array, GCC allocated that dynamic buffer at a lower stack address right below the saved return address. An upward buffer overflow from the dynamic region could then overwrite the saved return address without ever touching the canary.

Existing compiler testing approaches fail to catch such silent bugs due to two fundamental barriers in prior work. The first barrier of existing fuzzers is the **oracle gap**. Widely used compiler fuzzers (e.g., Csmith [2] and YARPGen [3]) check for compiler crashes or output mismatches across optimization levels. Because a binary with a broken defense still produces correct outputs on benign inputs, crash and differential oracles remain blind when a defense is weakened or broken. The second barrier in prior tools is locating defense-related compiler branches and satisfying their triggering conditions. Traditional directed fuzzers can reach given target branches, but they cannot tell which branches inside millions of lines of compiler code control a defense. Even after reaching a defense branch, exposing a silent failure requires satisfying defense-specific semantic conditions, such as opaque object bounds or cross-pass length propagation, rather than generic control-flow or data-flow coverage. For instance, in a GCC bug we uncovered (PR 125079 [4]), triggering silent `_FORTIFY_SOURCE` relaxation in the `strlen` pass requires pairing `__strcat_chk(dst, src, objsize)` on an unknown-length buffer with a subsequent `strlen(dst)` call that triggers the rewrite into `__stpcpy_chk`. Random mutators rarely synthesize such coupled defense semantics. Meanwhile, building static analyzers demands heavy manual effort from experts to craft custom rules for each defense mechanism and target ISA, while unconstrained LLM agents

suffer from severe hallucinations and false bug reports.

These barriers raise a central question: how can we systematically detect silent defense failures when neither the buggy compiler nor direct LLM judgment provides a reliable oracle? To answer this question, we present **DEFUZZ**, an invariant-directed agentic framework for testing compiler-instrumented defenses. Our core insight is that although defense implementations vary across backends, their intended security guarantees follow target-independent *security invariants*. DEFUZZ translates these invariants into programmatic bug oracles implemented as ISA-specific static and dynamic checkers, closing the oracle gap. Because LLM-assisted translation can introduce errors, DEFUZZ formalizes each invariant in a predicate IR with explicit applicability and safety semantics, deterministically lowering validated specifications into executable checkers. To trigger defense-critical logic, DEFUZZ maps each invariant to its corresponding defense-related compiler branches and guides LLM agents to synthesize inputs that satisfy both branch conditions and invariant preconditions. All bug verdicts remain strictly with deterministic checkers, avoiding both the blind spots of random mutators and LLM adjudication hallucinations.

Building DEFUZZ requires solving three main challenges. The first challenge is establishing reliable security invariants: because the compiler itself has bugs, we cannot trust it. Relying on a compiler's own behavior or implementation code as the ground truth creates circular self-validation. Our key idea is to separate intended security contracts from buggy compiler implementations by anchoring invariants in external ISA manuals, documented design assertions, and confirmed security patches, while abstracting past fixes into transferable failure patterns across defenses. The second challenge is checking these invariants with high accuracy across diverse ISAs, where distinct registers and stack layouts cause direct LLM judgment to hallucinate. DEFUZZ addresses this by formalizing each invariant in a predicate IR parameterized by target ISA specifications, lowering it into ISA-specific static and dynamic checkers. Dual static-dynamic cross-checking and differential baseline builds further verify each target, ensuring reliable verdicts with **zero LLM involvement in bug adjudication**. The final challenge is steering test generation to both locate defense branches and satisfy their semantic preconditions without being stripped by early optimizations. DEFUZZ maps each invariant to its controlling compiler passes and branches via call-graph and AST analysis, inserting compile-time probes to trace branch execution. Guided by branch traces and invariant preconditions, LLM agents iteratively synthesize and mutate seeds to expose deep defense flaws.

We evaluated DEFUZZ on GCC and LLVM across x86-64, AArch64, and RISC-V, spanning diverse defenses such as stack canaries, CFI, shadow stacks, register scrubbing, and `_FORTIFY_SOURCE`. DEFUZZ extracted 915 validated security invariants and transferred historical failure patterns across defense mechanisms and backends. During fuzzing, branch-guided seed synthesis hit substantially more target defense branches than unguided baselines, while our formally specified checkers identified violations with high accuracy and avoided the false reports produced by unconstrained LLM agents. Overall, DEFUZZ discovered **126 previously unknown silent failures in compiler-instrumented defenses**, of which **34 have already been confirmed upstream, with 11 fixed** (Figure 1), including bugs rated as high-severity vulnerabilities. These results show that DEFUZZ effectively uncovers critical silent defense bugs that evade existing compiler testing techniques.

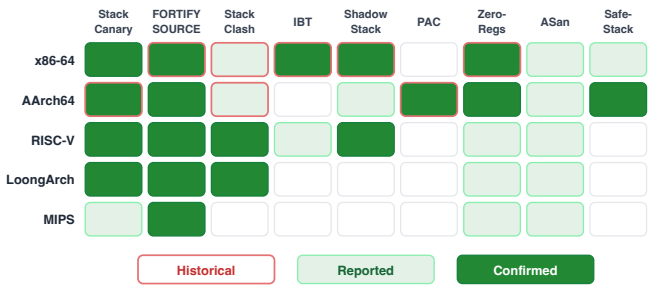


Fig. 1. Partial matrix of silent compiler defense bugs across representative defense mechanisms and target ISAs.

In answering this central question, this paper makes the following contributions:

- **Automated Invariant Extraction:** We decouple security contracts from buggy compiler outputs by extracting security invariants from ISA specifications, design assertions, and historical bug fixes, transferring failure patterns across defenses.
- **Validated Translation into Multi-ISA Programmatic Oracles:** We formalize security invariants in a predicate IR and lower them into formally specified static and dynamic checkers across target ISAs, verifying compiled code with high accuracy and zero LLM involvement in bug adjudication.
- **Branch-Guided Agentic Fuzzing:** We map invariants to defense-related compiler passes and branches, and guide specialized LLM agents with compile-time branch traces to synthesize seeds that satisfy defense triggering conditions.
- **New Silent Defense Bug Discovery:** We evaluate DEFUZZ on GCC and LLVM across multiple ISAs and discover 126 previously unknown silent failures in compiler-instrumented defenses, with 34 confirmed upstream, including 11 fixed.

II. MOTIVATION

A. Compiler-Instrumented Defenses

Compilers deploy automated defenses to protect software against security threats. Unlike standalone binary rewriters, compiler-instrumented defenses span both middle-end and backend stages of compilation. Middle-end passes analyze object bounds and control flow to insert or optimize high-level checks on the intermediate representation (IR). Backend passes enforce low-level protections by controlling stack frame

layouts, register assignments, and machine instructions across target ISAs.

Preserving security guarantees across this multi-stage pipeline is error-prone. A middle-end optimization can rewrite a defense check under flawed size assumptions, or a backend pass can misplace a stack guard slot. When such bugs occur, the compiler neither crashes nor emits warnings. Instead, it produces a working binary whose security protection is quietly weakened or broken.

B. Motivating Example

Listing 1 shows a silent defense failure in GCC's `_FORTIFY_SOURCE` implementation (PR 125079 [4]) discovered by DeFuzz. This bug had remained hidden for 14 years before upstream maintainers fixed it in the GCC mainline. Under `-D_FORTIFY_SOURCE=2`, GCC replaces unsafe string calls such as `strcat(dst, src)` on the 64-byte buffer `dst` with the bounds-checked builtin `__strcat_chk(dst, src, 64)`. At runtime, this builtin checks that the existing string in `dst` plus the appended `src` fits within 64 bytes, aborting execution on overflow.

```
1  char dst[64];
2
3  size_t append_and_measure(const char *src) {
4    init_opaque(dst); // hides prefix len k (k = 8)
5
6    // 1. Fortified call: __strcat_chk(dst, src, 64)
7    strcat(dst, src);
8
9    // 2. get_string_length() rewrites call to:
10   // __stpcpy_chk(dst + k, src, 64)
11   // BUG: bound stays 64 instead of 64 - k!
12   return strlen(dst);
13 }
```

Listing 1. Silent `_FORTIFY_SOURCE` relaxation in GCC (PR 125079).

In Listing 1, `init_opaque(dst)` initializes the 64-byte buffer `dst` with an 8-byte string ($k = 8$) whose length is unknown at compile time, followed by `strcat(dst, src)` and `strlen(dst)`. Executing `strcat(dst, src)` and `strlen(dst)` naively would scan `dst` twice: once to find the end of the prefix, and again to measure the combined string. To avoid this redundant scan, GCC's `tree-ssa-strlen` pass invokes `get_string_length()` in `tree-ssa-strlen.cc` when it processes the trailing `strlen(dst)` call. Inside `get_string_length()`, the `BUILT_IN_STRCAT_CHK` branch rewrites the earlier `__strcat_chk(dst, src, 64)` into `__stpcpy_chk(dst + k, src, 64)`, which copies `src` to `dst + k` and returns the end pointer to compute `strlen(dst)` directly. However, while GCC advances the destination pointer by k bytes, it fails to subtract k from the size bound, passing 64 instead of $64 - k = 56$. As a result, the runtime check still permits writing 64 bytes starting from `dst + 8`, allowing an 8-byte buffer overflow without triggering an abort.

C. Limitations of Existing Approaches

Existing compiler testing tools fail to detect this vulnerability due to two fundamental barriers in prior work. The first barrier of existing fuzzers is the **oracle gap**. Widely used compiler fuzzers such as Csmith [2], YARPGen [3], and EMI [5] only check whether the compiler crashes or whether compiled binaries produce consistent outputs on benign inputs. In Listing 1, GCC compiles the code cleanly, and any benign string `src` produces the exact same return value across all optimization levels. Because crash and differential testing do not check whether the binary enforces the reduced bound $(64 - k)$, they remain blind when the defense is weakened.

The second barrier in prior tools is locating defense-related compiler branches and satisfying their triggering conditions. Traditional directed fuzzers can steer execution toward given target branches, but they cannot tell which branches inside millions of lines of compiler source code control a defense, such as the `BUILT_IN_STRCAT_CHK` branch in `get_string_length()`. Even after reaching that branch, triggering the bug in Listing 1 requires three coupled conditions on `dst`: the buffer must have a known allocation size so GCC emits `__strcat_chk(dst, src, 64)`, its prefix length k must remain unknown at the append site so folding is deferred, and a subsequent `strlen(dst)` call must query the length to trigger the rewrite into `__stpcpy_chk(dst + k, src, 64)`. Random mutators rarely chain an opaque initialization, a fortified append, and a subsequent `strlen` call on the same buffer.

Beyond fuzzing, static analysis requires heavy manual effort from human experts to craft custom rules for each defense mechanism and target ISA. Meanwhile, using LLMs to audit compiler source code end-to-end causes severe hallucinations and false bug reports. To overcome these limitations, a testing framework must locate defense-related compiler branches and guide seed generation toward them. The generated seeds must satisfy the branch conditions so the compiler emits the defense code in the binary. The framework must then run the compiled binary with defense-triggering inputs and use deterministic oracles to check whether the protection is silently weakened or broken.

III. Design of DeFuzz

In this section, we present the design of DeFuzz, an invariant-directed framework for discovering silent failures in compiler-instrumented defenses. Figure 2 illustrates its overall workflow, which consists of four core steps: invariant generation, compiler source mapping, programmatic oracle construction, and agentic directed fuzzing. First, we extract security invariants from compiler source code, official documentation, and historical bug reports using a hybrid CoT and RAG pipeline. With these invariants established, we map them to the relevant compiler passes and decision branches, identifying where defenses are implemented and which conditions govern their execution. Next, we formalize these invariants in a predicate IR and lower them into programmatic bug oracles. We implement these oracles as static

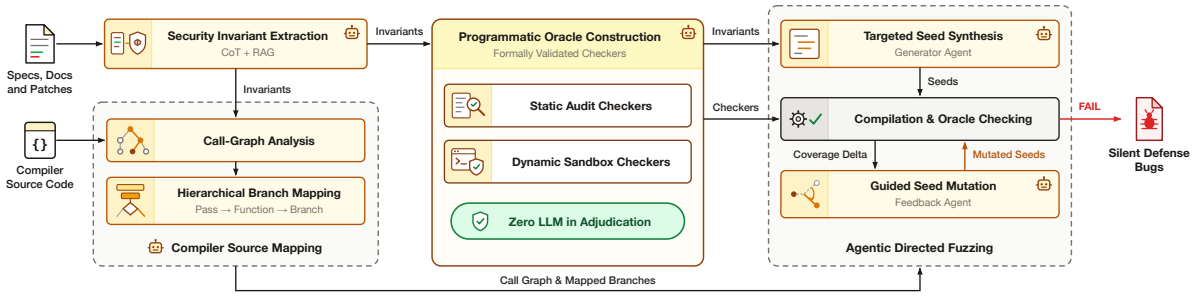


Fig. 2. Overview of DEFUZZ across four stages: (1) extracting security invariants via CoT and RAG, (2) mapping invariants to compiler branches, (3) formalizing invariants in a predicate IR to synthesize multi-ISA checkers, and (4) branch-guided agentic fuzzing.

and dynamic checkers across target ISAs to verify whether compiled binaries preserve intended defenses, avoiding false reports from model hallucination. Finally, our agentic directed fuzzing coordinates multiple specialized agents: a generator agent synthesizes program seeds designed to violate target invariants, while a feedback agent analyzes execution feedback to guide iterative mutations, with our programmatic oracles evaluating and verifying each candidate. We discuss each step in detail in the following subsections.

A. Security Invariant Generation

To detect silent defense failures, DEFUZZ extracts security invariants that specify the protection rules each compiler defense must enforce. Because the compiler itself may contain bugs, neither its implementation logic nor its compiled outputs can serve as the ground truth. Treating current compiler behavior as the reference would cause circular self-validation whenever a bug silently weakens or breaks a defense. Instead, DEFUZZ extracts security invariants from three sources: official compiler documentation and ISA/ABI manuals, developer comments and assertions in compiler source code, and confirmed historical bug reports.

Feeding whole compiler files or manuals to an LLM exceeds context limits and causes hallucinations. To keep each prompt focused, DEFUZZ splits compiler source code and official documentation into small, self-contained slices. For compiler source code, each slice contains a single C/C++ function with its leading header comment, or a single machine-description pattern (`define_insn` or `define_expand`). For official documentation, each slice contains a single manual section, such as a GCC Texinfo `@node` entry (e.g., `invoke.texi` and `tm.texi`) or an LLVM/Clang design section (e.g., `ControlFlowIntegrity.rst` and `LangRef.md`). Every slice records its file path, line numbers, defense mechanism, and target ISA. DEFUZZ then analyzes one slice at a time with CoT prompting [6] to extract the triggering precondition, the protection rule, and the observable violation signal. In the fortified string bug, reviewing the slice for `get_string_length()` and its header comment leads the model to infer a clear rule: when advancing a destination pointer by a prefix length, the pass must subtract that same length from the remaining size bound.

Different defenses and backends often perform similar low-level operations, such as updating size bounds, laying out stack frames, or clearing registers before return. As a result, new silent failures often share the same root causes as historical bugs. To find such bugs across defenses and architectures, DEFUZZ uses a hybrid RAG pipeline [7], [8] to transfer historical bug patterns. Searching directly with a raw bug patch only matches the original buggy function. Instead, DEFUZZ strips defense-specific function and flag names from each historical bug report, distilling a general query with three parts: a root-cause summary, defense-agnostic action keywords, and original defense terms to exclude, as shown in Listing 2 for GCC PR 96191 [9]. Using this query, DEFUZZ searches the sliced source and documentation corpus by combining BM25 keyword search with dense vector retrieval via Reciprocal Rank Fusion. BM25 matches underscore-split identifier tokens that share naming conventions across passes, such as `ix86_zero_call_used_regs`. Dense vector retrieval finds slices that implement the same operation with different data structures, such as the register bitmap in ARM's `cmse_nonsecure_entry_clear_before_return`. After filtering out slices that contain the excluded terms, DEFUZZ selects the top-k candidate slices. For each candidate slice, the model pairs the code with the historical root cause and uses CoT reasoning to derive a new invariant for that target defense.

```
# Distilled Query from GCC PR 96191
root_cause: "Function exit path returns without
             clearing caller-visible registers"
tokens:     ["epilogue", "caller_saved", "clear",
             "register", "return"]
exclude:    ["stack-protector", "canary"]
# Top Cross-Defense Hits via BM25 + Dense RRF
hit_1_bm25:  "config/i386/i386.cc"
  fn:  "ix86_zero_call_used_regs"
  why: "sub-tokens {zero, call, regs}"
hit_2_dense: "config/arm/arm.cc"
  fn:  "cmse_nonsecure_entry_clear_before_return"
  why: "root cause on register-clear bitmap"
```

Listing 2. RAG query structure and cross-defense retrieval hits.

All invariants from CoT analysis and RAG transfer are stored as structured records in a unified catalog, as shown in Listing 3. Each record tags the target defense and applicable

ISAs, followed by three core fields: the triggering precondition, the protection rule, and the observable violation signal. Because these records describe security rules without tying them to specific compiler files or functions, the same invariant can guide source mapping across different compilers and build binary checkers across multiple ISAs.

```
id:      "INV-FORT-014"
defense: "_FORTIFY_SOURCE"
isas:    ["x86_64", "aarch64", "riscv64"]
precond: "Fortified append on sized buffer N with
          prefix len k, followed by length query"
rule:    "Rewriting append to offset dst + k must
          reduce remaining size bound to N - k"
signal:  "Bound argument stays > N - k, or runtime
          write past N bytes fails to abort"
```

Listing 3. Structured security invariant record in the catalog.

B. Compiler Source Mapping

With the invariant catalog established, DEFUZZ maps each security invariant to the compiler passes and decision branches that govern its target defense. An invariant specifies what protection a compiled binary must uphold, yet it does not indicate where the compiler implements that logic across millions of lines of C and C++ source code. Lexical search alone is insufficient because a single defense often spans middle-end optimization passes, target hooks, and backend machine descriptions, while providing entire source trees exceeds model context limits. Instead, DEFUZZ equips a mapping agent with a symbol indexer, a call-graph analyzer, and an AST parser. Using these deterministic tools, the agent narrows the codebase along a three-level hierarchy of passes, functions, and conditional branches.

Starting from command-line defense flags or fortified builtin names, the agent queries the symbol indexer and call graph to identify the middle-end or backend pass responsible for the protection. Within that pass, it traverses call-graph edges to locate functions that compute object bounds, lay out stack frames, or insert runtime checks. It then parses each function AST to isolate the conditional expressions and state variables that gate defense emission. Listing 4 shows the resulting mapping for `INV-FORT-014`, tracing fortified string calls to `tree-ssa-strlen`, `get_string_length()`, and the `BUILT_IN_STRCAT_CHK` rewrite branch. Applying the same hierarchy to stack canary invariants maps frame layout rules to `pass_stack_protect`, `expand_used_vars()`, and dynamic allocation predicates such as `has_vla`.

```
# Three-Level Hierarchy Mapping
pass:     "tree-ssa-strlen"
  file:   "gcc/tree-ssa-strlen.cc"
function: "get_string_length"
branch:   "case BUILT_IN_STRCAT_CHK"
  vars:   ["elen", "objsize", "lhs"]

# Injected Compile-Time Branch Probe
DEFUZZ_HIT("fortify#b4e1:hit");
```

Listing 4. Source mapping and branch probe for `INV-FORT-014`.

From each validated mapping, DEFUZZ deterministically instruments a copy of the compiler source tree by inserting a lightweight `DEFUZZ_HIT` probe at each mapped branch, as shown in Listing 4. Each probe carries a unique identifier encoding the defense mechanism and target branch, and DEFUZZ compiles the modified tree into instrumented drivers `GCC_INST` and `CLANG_INST`. Probe logging remains dormant during normal builds and activates only when the `DEFUZZ_LOG` environment variable is set by the fuzzer, writing reached branch identifiers to standard error. Because probes execute strictly inside the compiler process at build time, they never alter the emitted machine instructions, stack layout, or runtime behavior of compiled binaries.

C. Programmatic Oracle Construction

To verify whether compiled binaries preserve intended defenses, DEFUZZ translates security invariants from the catalog into programmatic bug oracles implemented as static and dynamic checkers across target ISAs. Online bug verdicts are decided strictly by these compiled checkers, with **zero LLM involvement in bug adjudication**. However, translating invariants into executable checkers offline still relies on LLM assistance, creating a risk of translation errors. Generating raw Go or C analysis code from scratch requires hundreds of lines of parsing boilerplate per rule. Minor coding mistakes, such as matching mnemonics inside assembly comments, inverting stack offset inequalities, or omitting caller-saved scratch registers on unfamiliar backends, directly cause false alarms or missed bugs.

Observation Primitives and Target Specifications. DEFUZZ prevents low-level coding errors by separating declarative check logic from binary parsing and hardware conventions. For a seed compiled on architecture i, deterministic **observation primitives** Ω extract a structured observation state $\omega = \langle \omega_{\text{st}}, \omega_{\text{dyn}} \rangle$ from build artifacts. Static primitives inspect compiler dumps, disassembly, and ELF sections to recover imported symbols, comment-stripped instruction bodies, stack frame offsets, and fortified call arguments. Dynamic primitives execute the binary inside an isolated sandbox to capture post-return register snapshots and boundary overflow signals, while supporting optional Go helper bindings for specialized checks. To handle backend variations without rewriting rules per architecture, DEFUZZ pairs Ω with a read-only **target specification table** $\mathcal{T}_i$, audited once against official ABI manuals. Table I summarizes representative fields of $\mathcal{T}_i$ across x86-64, AArch64, and RISC-V.

Formal Check IR and Four-State Semantics. Rather than generating raw binary parsers, DEFUZZ formalizes each invariant in a domain-specific predicate IR, termed **Check IR**, over ω and $\mathcal{T}_i$:

$$\phi = \langle \mu,\ I_\phi,\ G_{\text{app}}(\omega, \mathcal{T}_i),\ P_{\text{safe}}(\omega, \mathcal{T}_i) \rangle \tag{1}$$

Here, $\mu \in \{\mathsf{Static}, \mathsf{Dynamic}\}$ selects the inspection modality, I_ϕ lists applicable target ISAs, G_{app} tests whether the compiled binary activates the defense precondition, and P_{safe} asserts

TABLE I
SELECTED HARDWARE AND ABI PARAMETERS IN THE TARGET SPECIFICATION TABLE $\mathcal{T}_i$.

Field	x86-64	AArch64	RISC-V
δ_{grow}	-1 (downward)	-1 (downward)	-1 (downward)
ra_kind	Stack slot	Link reg `x30`	Link reg `ra`
R_{caller}	`rax–r11`	`x0–x18`	`t0–t6, a0–a7`
r_{arg4}	`rcx`	`x3`	`a3`
pad_bytes	`f30f1efa` (`endbr64`)	`5f2403d5` (`bti`)	$\perp$
$\text{align}_{\text{insn}}$	1 byte	4 bytes	2 bytes

the protection property using quantifier-free arithmetic and bounded set predicates. Given observation ω on architecture i, the oracle evaluates ϕ into one of four deterministic verdicts:

$$\text{Eval}(\phi, \omega, \mathcal{T}_i) = \begin{cases} \textbf{ERROR}, & \text{if build or parse fails} \\ \textbf{NA}, & \text{if } i \notin I_\phi \ \vee \ \neg G_{\text{app}} \\ \textbf{PASS}, & \text{if } G_{\text{app}} \ \wedge \ P_{\text{safe}} \\ \textbf{FAIL}, & \text{if } G_{\text{app}} \ \wedge \ \neg P_{\text{safe}} \end{cases} \tag{2}$$

This four-state contract eliminates two major sources of false positives in compiler testing. Build failures or malformed dumps map to **ERROR** rather than spurious bug claims, while seeds that do not activate the target defense precondition evaluate to **NA** rather than misleading passes.

Formal IR Validation and Differential Anchoring. Before any Check IR rule enters the fuzzing loop, DEFUZZ validates ϕ through two deterministic offline gates without LLM involvement. The first gate enforces *formal well-formedness* over ϕ, verifying that all predicate expressions are closed, every observation primitive in Ω respects its signature and inspection modality μ, and every referenced hardware parameter is defined in $\mathcal{T}_i$ across all $i \in I_\phi$. Once ϕ passes well-formedness validation, the second gate deterministically lowers it into a standalone Go checker and runs *differential baseline anchoring* on minimal probe seeds. Each lowered checker must evaluate to **PASS** on hardened builds such as `-fstack-protector-all` or `-D_FORTIFY_SOURCE=2`, and evaluate to **FAIL** or **NA** on unhardened or fault-seeded builds, filtering out ill-formed or misfiring rules before online fuzzing.

Static and Dynamic Cross-Checking. During online evaluation, DEFUZZ runs fast static checkers before sandboxed dynamic checkers while sharing cached binary observations across rules. Static checkers resolve structural properties directly from ELF metadata and disassembly. For instance, forward-edge CFI checkers read $\mathcal{T}_i$.pad_bytes and $\mathcal{T}_i.\text{align}_{\text{insn}}$ to scan indirect-callable functions for missing entry landing pads or unintended interior `endbr64` and `bti` gadgets across x86-64 and AArch64, while returning **NA** when pad_bytes $= \perp$.

When defense correctness depends on runtime execution state, dynamic checkers run compiled harnesses inside an isolated sandbox. Consider stack canary epilogue scrubbing in GCC PR 96191 [9], where vulnerable functions compare the stack canary in a scratch register without clearing that register before return:

$$\begin{aligned} G_{\text{app}}(\omega, \mathcal{T}_i) &: \ \texttt{__stack_chk_fail} \in \omega.\text{imports} \\ P_{\text{safe}}(\omega, \mathcal{T}_i) &: \ \forall r \in \mathcal{T}_i.R_{\text{caller}}, \ \omega.\text{post_regs}[r] \neq \omega.\text{canary} \end{aligned} \tag{3}$$

Listing 5 shows the lowered post-return register probe. Reading $\mathcal{T}_i.R_{\text{caller}}$ automatically inspects `rax` through `r11` on x86-64, `x0` through `x18` including intra-procedure temporaries on AArch64, and `t0` through `a7` on RISC-V without modifying the rule.

Similarly, for the fortified string rewrite, static inspection checks the bound register $\mathcal{T}_i.r_{\text{arg4}}$ at `__stpcpy_chk` call sites for degenerated `UINT64_MAX` bounds, whereas dynamic probing verifies runtime prefix arithmetic. Listing 6 shows the lowered dynamic harness, which initializes a 64-byte buffer with an 8-byte opaque string and appends a 60-byte payload via `__strcat_chk` followed by `strlen`. Because the 68-byte total write exceeds the 64-byte buffer, an intact defense must raise `SIGABRT` and evaluate to **PASS**, whereas GCC's unadjusted bound of 64 permits the overflow to finish and triggers **FAIL**.

```c
void audit_canary_leak(uintptr_t canary) {
  uintptr_t r0, r1, r2 /* ... */;
  asm volatile(
    "mov %%rax, %0; mov %%rcx, %1; mov %%rdx, %2"
    // ... expanded from T_i.R_caller per ISA
    : "=r"(r0), "=r"(r1), "=r"(r2)
  );
  if (r0 == canary || r1 == canary /* ... */)
    emit_verdict(FAIL, "Canary leaked");
  else
    emit_verdict(PASS, "Canary cleared");
}
```

Listing 5. Lowered post-return register probe for canary scrubbing.

```c
void test_fortify_strcat(const char *src60) {
  char buf[64];
  init_opaque_8bytes(buf); // opaque prefix k = 8
  setup_sigabrt_handler(); // trap fortify abort

  // Rewritten to __stpcpy_chk(buf + 8, src60, 64)
  __builtin___strcat_chk(buf, src60, sizeof(buf));
  sink = __builtin_strlen(buf); // triggers fold
  emit_verdict(FAIL, "Overflow unchecked");
}

void sigabrt_handler(int sig) {
  emit_verdict(PASS, "Overflow trapped");
  _exit(0);
}
```

Listing 6. Dynamic boundary probe for `_FORTIFY_SOURCE` bounds.

D. Agentic Directed Fuzzing

With the invariant catalog, mapped compiler branches, and validated checkers in place, DEFUZZ launches its online directed fuzzing campaign, summarized in Algorithm 1. Allowing unconstrained LLM agents to explore compiler codebases freely leads to nondeterministic tool loops, lost provenance, and unverified bug claims. Instead, DEFUZZ enforces a strict

Algorithm 1 Agentic Directed Fuzzing per Invariant

Require: Invariant ϕ, mapped path $\mathcal{M}_\phi$, spec $\mathcal{T}_i$, budget B
Ensure: Minimized bug PoCs $\mathcal{F}$ on versioned blackboard $\mathcal{B}$

$\mathcal{F} \leftarrow \emptyset$; $g \leftarrow \mathsf{InitGuidance}(\phi, \mathcal{M}_\phi)$
for $t = 1$ **to** B **do**
  $s_t \leftarrow \mathsf{GeneratorAgent}(\phi, \mathcal{M}_\phi, g)$
  $(b_t, \tau_t) \leftarrow \mathsf{CompileInst}(s_t, i)$ {`DEFUZZ_LOG=1`}
  $v_t \leftarrow \mathrm{Eval}(\phi, \Omega(b_t, s_t), \mathcal{T}_i)$
  $\mathcal{B}.\mathsf{Append}(s_t, \tau_t, v_t)$
  if $v_t =$ **FAIL then**
    $s^* \leftarrow \mathsf{MinimizerAgent}(s_t, \phi, \mathcal{T}_i)$
    $\mathcal{F} \leftarrow \mathcal{F} \cup \{s^*\} \cup \mathsf{Amplify}(s^*, \phi, I_\phi)$
    break
  else if $\mathsf{HitTarget}(\tau_t, \mathcal{M}_\phi) \wedge \mathsf{Stalled}(\mathcal{B})$ **then**
    $\phi \leftarrow \mathsf{RefineAndValidate}(\phi, \mathcal{B})$; $\mathsf{Replay}(\phi, \mathcal{B})$
  else
    $g \leftarrow \mathsf{FeedbackAgent}(\phi, \mathcal{M}_\phi, \tau_t, v_t)$
  end if
end for
return $\mathcal{F}$

topological workflow through a deterministic orchestrator. At startup, the orchestrator loads validated invariants into a queue and allocates a fixed iteration budget B per target invariant and architecture. Each round executes a fixed sequence of targeted seed synthesis, compilation with `GCC_INST` or `CLANG_INST`, two-phase oracle evaluation, and trace-guided mutation. All intermediate artifacts, including synthesized seeds, compiler flags, `DEFUZZ_HIT` branch traces, and oracle verdicts, are recorded on an append-only versioned blackboard $\mathcal{B}$, enabling deterministic replay of every test seed and bug verdict.

Branch-Guided and Optimization-Resistant Seed Synthesis. In each round, the Generator Agent synthesizes C or C++ seeds designed to reach the mapped path $\mathcal{M}_\phi$ and satisfy the triggering precondition of ϕ. Conventional random program generators fail at two stages. Early middle-end passes such as Dead Code Elimination, constant folding, and dead-store removal routinely strip unused buffers and redundant calls before defense passes execute. Even when code survives early cleanup, exposing a silent defense flaw requires coupled semantic constraints across multiple statements. In the fortified string bug, triggering the faulty rewrite inside `get_string_length()` requires combining a statically sized destination buffer with a runtime-opaque prefix length, a fortified `strcat` append, and a trailing `strlen` query on the same buffer. Guided by $\mathcal{M}_\phi$ and ϕ, the Generator Agent constructs compact multi-statement programs satisfying these coupled conditions. Rather than applying blanket `volatile` qualifiers that would suppress the target optimization pass itself, the agent places opacity barriers selectively at function boundaries. It derives dynamic allocation sizes from runtime `argc` inputs, marks external helper and sink functions with `noinline` and `noclone` attributes, and inserts inline `asm` memory clobbers to keep defense-relevant stack frames and bounds live through backend lowering.

Trace-Driven Divergence Feedback and Oracle Refinement. Each synthesized seed is compiled by the instrumented driver with `DEFUZZ_LOG` enabled, producing a candidate binary b_t alongside a compile-time trace τ_t of reached defense branches. When $\mathrm{Eval}(\phi, \omega, \mathcal{T}_i)$ returns a non-failing verdict, DEFUZZ passes compiler diagnostics, checker evidence, and τ_t to the Feedback Agent, which diagnoses three distinct failure modes. If evaluation returns **ERROR**, the agent repairs invalid target flags, missing declarations, or ABI type mismatches. If evaluation returns **NA** or **PASS** while target branches are absent from τ_t, the agent compares τ_t against the mapped hierarchy $\mathcal{M}_\phi$ to pinpoint where compilation diverged. For instance, τ_t may reveal that compilation entered `expand_used_vars()` with `has_vla` unset, or returned from `get_string_length()` prior to `BUILT_IN_STRCAT_CHK`. From this divergence, the Feedback Agent issues structural mutation directives specifying which variable qualifiers, buffer declarations, or call sequences the Generator Agent should adjust next. When repeated rounds hit the deepest target branch in $\mathcal{M}_\phi$ while still returning **NA**, the orchestrator triggers bounded Check IR refinement on G_{app}, re-validates the updated rule through the offline gates, and replays recorded seeds from $\mathcal{B}$.

Oracle-Locked Minimization and Lateral Amplification. When a checker returns **FAIL**, the oracle confirms a silent defense failure from deterministic machine evidence alone. DEFUZZ then invokes a Minimizer Agent that couples C-Reduce [10] with the programmatic oracle to isolate a minimal root-cause trigger. Standard reduction predicates based on compiler crashes or exit codes often collapse silent defense bugs into unrelated undefined behavior. Instead, DEFUZZ recompiles every reduced candidate with the instrumented driver and accepts a reduction step only if the target checker still evaluates to **FAIL** while preserving the target branch hit in τ. Once minimization produces a clean Proof-of-Concept seed s^*, the orchestrator executes a bounded lateral amplification step before advancing to the next invariant. Conditioning on s^*, the Generator Agent systematically varies surrounding data types, struct nesting, alignment attributes, optimization levels, sibling builtins such as `__strcpy_chk` and `__mempcpy_chk`, and peer target architectures in I_ϕ to uncover related silent defense flaws sharing the same root cause.

IV. IMPLEMENTATION

The DeFuzz architecture is decoupled into a high-performance evaluation core and a Python-based agentic orchestrator. The core service, implemented in Go, provides deterministic build execution, coverage tracking, and oracle adjudication. It exposes these capabilities to the orchestrator via gRPC for high-throughput programmatic access, and via the Model Context Protocol (MCP) to provide agents with read-only tools for source code search, invariant querying, and delta debugging (C-Reduce) invocation. Crucially, the oracle definitions, static/dynamic checkers, and ISA applicability rules are centralized within the Go core, serving as the single source of truth for all metadata.

The orchestrator governs the explicit agentic loop, maintaining the strict topological order of generation, routing, build,

and oracle adjudication. It manages the versioned blackboard, which tracks the lineage of seed corpora, cumulative coverage metrics, and feedback guidance. To ensure rigorous auditability, every execution persists a dedicated artifact bundle containing a serialized state chain, full toolchain configurations, and LLM hyperparameters. This design enables deterministic replay and post-hoc attribution of any confirmed bug claim.

V. Evaluation

We evaluate DeFuzz on GCC and LLVM across multiple target architectures and compiler-instrumented defenses. Our experiments answer four research questions:

- **RQ1 (Bug Discovery):** Can DeFuzz uncover previously unknown, high-severity silent defense bugs across compilers, defense families, and architectures?
- **RQ2 (Invariant and Oracle Quality):** How effectively does DeFuzz extract grounded security invariants and reject faulty checkers via offline gates?
- **RQ3 (Ablation Study):** How much does each design choice, starting with formal Check IR synthesis, improve bug detection and precision?
- **RQ4 (Telemetry and Overhead):** What are the agent tool-call patterns, token costs, KV-cache hit rates, and build overheads across pipeline stages?

A. Experimental Setup

Target Compilers and Architectures. We evaluate DeFuzz on release builds of GCC 16.2.0 (verified against GCC 17.0 trunk) and LLVM/Clang 23.1.2. Each campaign targets four primary 64-bit architectures: x86-64, AArch64, RISC-V (`rv64gc`), and LoongArch64. For portable target hooks, DeFuzz also cross-compiles seeds for 32-bit x86, ARM, MIPS64, PowerPC64LE, and s390x.

Evaluated Defense Families. Table II groups the audited mechanisms into six defense families spanning both middle-end and backend compiler stages. Control-flow defenses cover IBT, BTI, PAC, KCFI, and RISC-V `Zicfilp`. Stack defenses cover stack canaries, stack-clash probing, SafeStack, CET shadow stacks, ARM GCS, `Zicfiss`, and ShadowCallStack. Memory sanitizers include ASan, HWASan, MTE stack tagging, and UBSan. State-scrubbing and speculation defenses include `-fzero-call-used-regs`, `-fstrub`, SLH, SLS, LVI, and retpolines. The remaining two families cover object-bounds fortification (`_FORTIFY_SOURCE`, `counted_by`, `-fstrict-flex-arrays`, `-fanalyzer`) and hardened calls or enclaves (RELRO, `noplt`, `musttail`, SGX).

Environment and Model Configuration. All experiments run inside an isolated Linux container on a 16-vCPU ARM Neoverse-N2 server. The container bundles GNU and LLVM cross-toolchains, QEMU user-mode emulators for x86-64, AArch64, RISC-V, and LoongArch64, and C-Reduce [10]. All LLM agents query `deepseek-flash` with prefix KV-caching enabled. Hybrid RAG indexes 5,380 GCC and 1,030 LLVM slices using `BAAI/bge-small-en-v1.5` embeddings fused with BM25. Step 3 caps offline Check IR repair at three attempts per invariant. Step 4 allocates an online budget of $B = 8$ iterations per invariant and target ISA. All online bug verdicts are decided strictly by compiled checkers, with **zero LLM involvement in bug adjudication**.

Baselines and Controlled Variants. We compare DeFuzz against Csmith [2] and YARPGen [3] compiled under identical hardening flags from `-O0` to `-O3`. We also define five controlled ablations to isolate internal components. *w/o Formal Check IR* replaces Check IR with direct LLM synthesis of raw Go checkers. *w/o Hybrid RAG* disables historical bug transfer, and *w/o Source Mapping* removes AST branch mapping and `DEFUZZ_HIT` probes. *w/o Programmatic Oracle* replaces compiled checkers with LLM judgment, while *Bare Agent* runs an unconstrained tool-using agent.

B. RQ1: Silent Defense Bug Discovery

Overall Discovery and Upstream Triage. Table II summarizes our bug-finding results across GCC and LLVM. DeFuzz discovered **126 previously unknown silent defense bugs**, comprising 70 in GCC and 56 in LLVM. To respect maintainer review bandwidth, we filed 86 reports so far and staged 40 recent Check IR findings in four 10-bug batches. Among the filed reports, **34 have been confirmed upstream and 11 have already been fixed**. Several fixes were backported to release branches as old as GCC 14. Maintainer feedback also identified four intentional design choices, seven disputed language-boundary cases, and four independently patched trunk defects. We exclude all 15 cases from the 126 bug count. Under identical hardening flags, Csmith and YARPGen trigger **[TBD: N]** compiler crashes and functional miscompilations, yet both find **zero** silent defense bugs. Crash and differential testing check output equivalence on benign inputs, leaving them blind when compilers silently weaken or break a defense.

Stage, Architecture, and Severity Distribution. Figure 3 characterizes the root causes and affected architectures of all 126 bugs. As shown in Figure 3(a), backend target lowering and frame layout account for 41.3% (52/126) of all bugs. Middle-end IR passes account for 26.2% (33/126), while frontend attribute handling accounts for 13.5% (17/126). Another 17.5% (22/126) arise in driver option lowering or LTO serialization. In GCC PR 128276 [11], for example, LTO stream-in drops every HWASan and MTE stack-tagging scope marker across compilation units. Figure 3(b) shows that **56 bugs (44.4%) have High security severity** and 44 (34.9%) have Medium severity. These flaws bypass CFI landing pads, corrupt shadow stacks, or disable memory sanitizers without emitting any warnings. Half of all bugs (63/126) affect portable logic across three or more ISAs. Among the 63 architecture-restricted bugs, **57.1% (36/63) affect only non-x86 backends** (AArch64, RISC-V, and LoongArch64). Examples include broken RISC-V `Zicfilp` landing pads [12], premature RVV stack-clash CFA switches [13], LoongArch canary leaks [14], and AArch64 PAC/KCFI flaws [15], [16]. These results show that newer non-x86 defenses harbor dense silent failure modes missed by x86-only testing.

TABLE II
SUMMARY OF 126 PREVIOUSLY UNKNOWN SILENT DEFENSE BUGS DISCOVERED BY DEFUZZ ACROSS GCC AND LLVM.

Defense Family	Checkers (G/L)	GCC	LLVM	Total	Conf. (Fix)	Affected ISAs	Representative Upstream IDs
CFI & Landing Pads	201 (39/162)	10	23	33	11 (3)	x86, ARM64, RV64, LA64	PR 125084, 128360; #208751, #236673
Stack & Shadow Stacks	113 (62/51)	15	13	28	11 (4)	x86, ARM64, RV64, LA64, MIPS	PR 125049, 128271; #237864, #237865
Memory Sanitizers & MTE	179 (58/121)	8	13	21	3 (0)	x86, ARM64, RV64, LA64	PR 128276, 128280; #237909, #236685
Scrubbing & Speculation	121 (73/48)	14	7	21	4 (1)	x86, ARM64, RV64, Generic	PR 128287, 128388; #236795, #236796
Bounds & Fortification	130 (100/30)	17	0	17	2 (1)	x86, ARM64, RV64, LA64	PR 125079, 127333, 128274
Call, Link & Enclave	171 (142/29)	6	0	6	3 (2)	x86, RV64, LA64, Generic	PR 117992, 128015, 128283
DEFUZZ Total	**915 (474/441)**	**70**	**56**	**126**	**34 (11)**	**All evaluated ISAs**	**86 filed; 40 staged in 4 batches**
Csmith [2] (hardened)	–	0	0	0	0 (0)	x86, ARM64, RV64, LA64	**[TBD: N]** crashes; 0 silent bugs
YARPGen [3] (hardened)	–	0	0	0	0 (0)	x86, ARM64, RV64, LA64	**[TBD: N]** crashes; 0 silent bugs

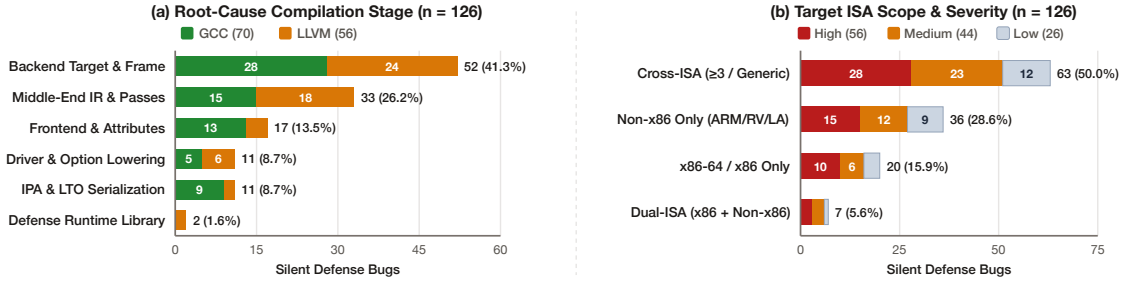


Fig. 3. Characterization of the 126 silent defense bugs discovered by DEFUZZ: (a) distribution across compiler stages in GCC and LLVM, and (b) target ISA scope broken down by security severity.

```c
// Compile: gcc -O2 -mshstk
void test_cet(long *cnt, void *tok) {
  long v = *cnt - 1; *cnt = v;
  _clrssbsy(tok); // clears hardware ZF = 0
  if (v == 0) on_zero();
}
// Emitted x86-64 assembly:
// subq $1, (%rdi) # sets ZF when *cnt == 0
// clrssbsy (%rsi) # hardware clears ZF = 0
// je on_zero # testq deleted; dead!
```

Listing 7. Case study 1 (GCC PR 128271, fixed in GCC 14–17): missing RTL clobbers on x86 CET shadow-stack intrinsics.

```c
// Flags: -O2 -march=armv8-a+sve -fsanitize=safe-stack
#include <arm_sve.h>
void sink(svint32_t *p);
void f(void) {
  svint32_t v = svdup_s32(-1); // width = VL bytes
  sink(&v); // escapes to unsafe
}
// Emitted AArch64 assembly:
// sub x9, x21, #16 // fixed 16-B unsafe slot
// str x9, [x20, x19] // updates unsafe_sp
// st1w {z0.s}, p0, [x21, x8, lsl #2] // VL B!
```

Listing 8. Case study 2 (LLVM #237864, confirmed): AArch64 SafeStack allocates a fixed 16-byte slot for scalable SVE locals.

Case Study 1: Silent CET Shadow-Stack Corruption in GCC. Listing 7 shows a high-severity x86 CET shadow-stack bug found by DEFUZZ (GCC PR 128271 [17]). Per the Intel SDM, `CLRSSBSY` clears a shadow-stack busy token, writes `CF`, and unconditionally clears `ZF` to zero. In addition, `WRSSQ` writes shadow-stack memory that ordinary `MOV` loads can read. From this specification, DEFUZZ extracted an invariant requiring shadow-stack intrinsics to preserve subsequent flag-dependent branches and post-write verification reads. In `gcc/config/i386/i386.md`, however, five CET patterns (`clrssbsy`, `@wrss<mode>`, `@wruss<mode>`, `saveprevssp`, and `rstorssp`) omitted `FLAGS_REG` and memory clobbers. As a result, RTL `compare-elim` deleted `testq` after `_clrssbsy` and reused stale flags from `subq`, making `je on_zero` permanently dead. In the same pass pipeline, RTL CSE folded post-`_wrssq` verification checks (`*ss == val`) to zero. GCC maintainers patched trunk within 24 hours and backported the fix to GCC 16, 15, and 14.

Case Study 2: AArch64 SafeStack Frame Overflow in LLVM. Listing 8 shows a high-severity flaw between SafeStack and ARM SVE in Clang 23.1.2 (LLVM #237864 [18]). Under `-fsanitize=safe-stack`, escaping stack buffers move to an unsafe stack to isolate return addresses on the safe stack. When DEFUZZ generated an escaping `svint32_t` local, `getStaticAllocaAllocationSize()` returned zero for the scalable vector type. `moveStaticAllocasToUnsafeStack()` then turned that zero size into a fixed 16-byte slot (`sub x9, x21, #16`), after which `st1w` wrote `VL` bytes to `usp - 16`. Running the binary under `qemu-aarch64` with `VL = 32` and `64` bytes overwrote 16 and 48 bytes of caller unsafe-stack data without any warning.

TABLE III
OFFLINE INVARIANT EXTRACTION, SOURCE MAPPING, AND TWO-GATE CHECK IR VALIDATION FUNNEL ACROSS GCC AND LLVM.

Offline Pipeline Metric	GCC	LLVM	Total
Modular source/manual slices reviewed	286	294	580
Segmented CoT invariants accepted	465	439	904
Hybrid RAG candidates retrieved	785	769	1,554
Hybrid RAG invariants accepted (0 overlap)	10	7	17
Step 1 Candidate Invariant Catalog	**475**	**446**	**921**
Mapped pass–function–branch paths ($\mathcal{M}_\phi$)	1,669	1,253	2,922
Instrumented source files (`GCC/CLANG_INST`)	342	405	747
First-attempt Check IR pass rate	84.0%	80.5%	82.3%
Gate 1 errors caught (well-formedness)	36	80	116
Gate 2 errors caught (differential anchoring)	48	41	89
Quarantined rules after 3 attempts	1	5	6
Step 3 Validated Oracles Entering Fuzzing	**474**	**441**	**915**

Compiling without `-fsanitize=safe-stack` sized the frame properly via `addvl`, and LLVM maintainers confirmed the miscompilation.

C. RQ2: Invariant Extraction and Formal Oracle Quality

Invariant Extraction and RAG Grounding. Table III tracks how DEFUZZ builds and validates its oracle catalog offline. In Step 1, segmented CoT analyzed 580 modular pass, target-hook, and manual slices (6.16M characters) across GCC and LLVM, extracting 904 security invariants. Hybrid RAG distilled 100 historical defense fixes into mechanism-neutral queries and retrieved 1,554 candidate slices. Origin-anchor exclusion and source grounding rejected 1,537 noisy or duplicate hits, accepting 17 cross-defense invariants with zero overlap against CoT. Step 1 produced **921 candidate invariants** (475 GCC, 446 LLVM), grounded in 671 compiler assertions or hooks and 250 specification rules. In Step 2, the mapping agent resolved all 921 invariants to **2,922** pass–function–branch paths across 747 source files with zero failures.

Two-Gate Check IR Validation Funnel. In Step 3, DEFUZZ formalized all 921 invariants into Check IR across 1,120 bounded authoring attempts (1.22 attempts per invariant). On the first attempt, **758 rules (82.3%) passed both deterministic validation gates** without human intervention. Across failed attempts, Gate 1 (formal well-formedness) caught 116 errors: 55 hallucinated primitives outside Ω, 55 malformed predicate expressions, and 6 unbound fields in $\mathcal{T}_i$. Gate 2 (differential baseline anchoring) caught another 89 errors, comprising 81 probe misfires between hardened and unhardened builds and 8 Go lowering violations. Exact gate diagnostics enabled the authoring agent to repair 157 of the 163 failed rules within three attempts, quarantining only 6 rules. In total, **915 validated programmatic oracles (99.3%)** entered online fuzzing, implemented as 903 static phase-1 checkers and 12 sandboxed dynamic phase-2 checkers.

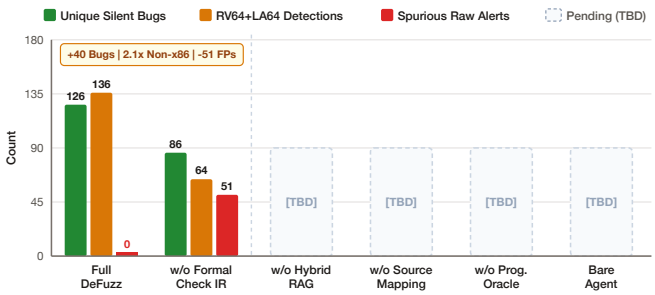


Fig. 4. Ablation comparison across DEFUZZ variants showing unique silent defense bugs, non-x86 (RISC-V and LoongArch64) detections, and spurious raw alerts.

TABLE IV
CONTROLLED ABLATION RESULTS ACROSS DEFUZZ VARIANTS. UNRUN COMPONENT ABLATIONS ARE MARKED AS **TBD**.

System Variant	1st-Pass	Tok.	RV+LA	FPs	Bugs
Full DEFUZZ (Check IR)	**82.3%**	**54.8M**	**136**	**0**	**126**
w/o Formal Check IR	70.5%	638.4M	64	51	86
w/o Hybrid RAG	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
w/o Source Mapping	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
w/o Prog. Oracle	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
Bare Agent	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**

D. RQ3: Ablation Study

Impact of Formal Check IR Synthesis. Figure 4 and Table IV compare full DEFUZZ against *w/o Formal Check IR*, where the agent writes standalone Go checkers directly from invariants. Across all 921 invariants, raw Go synthesis lowers first-attempt validation from 82.3% to 70.5%. It also triggers 98 Go compile or type errors versus 7 lowering errors under Check IR (**14.0× fewer**). Because raw Go agents repeatedly debug multi-hundred-line parsers, Step 3 token usage rises from 54.81M to 638.44M (**11.6**×) and authoring time grows from 2.20 h to 17.11 h (**7.8**×).

To isolate oracle quality from seed-generation randomness, we ran both checker suites on the same 547 candidate triggers (161 GCC, 386 LLVM) across four ISAs. Check IR raises verified failing candidates from 102 to 154 (**+51.0%**) and lifts total ISA-level detections from 188 to 315 (**1.68**×). Non-x86 backends benefit most: detections grow from 33 to 72 on RISC-V (**2.18**×) and from 31 to 64 on LoongArch64 (**2.06**×), reaching 136 versus 64 combined (**2.13**×). Among 104 triggers missed by raw Go checkers, 80 returned **NA** because hand-generated parsers hardcoded x86-64 or AArch64 mnemonics and bailed out on RISC-V or LoongArch64. Check IR avoids this gap by reading backend conventions directly from $\mathcal{T}_i$. Check IR also eliminates **51 spurious raw FAIL alerts** (12 GCC, 39 LLVM) caused by matching mnemonics inside assembly comments or inverting signed frame offsets. In total, Check IR uniquely uncovers **40 true silent defense bugs** (20 GCC, 20 LLVM, including 16 High-severity bugs), lifting total bug discovery from 86 to 126.

TABLE V
STAGE-BY-STAGE LLM API INVOCATIONS, TOKEN ACCOUNTING, PREFIX KV-CACHE HIT RATES, AND WALL-CLOCK RUNTIME ACROSS THE FULL GCC AND LLVM EVALUATION CAMPAIGNS.

Pipeline Stage	Phase	LLM Calls	Input (M)	Cached (Hit %)	Output (M)	Total (M)	Time (h)
Step 1: Security Invariant Extraction	Offline	–[†]	–	–	–	15.00	2.00
Step 2a: Compiler Source Mapping	Offline	35,523	617.07	572.84 (92.8%)	22.28	639.36	29.50
Step 2b: AST Branch Instrumentation	Offline	0	0.00	–	0.00	0.00	0.82
Step 3: Check IR Oracle Synthesis	Offline	2,733	47.35	36.37 (76.8%)	7.46	54.81	2.20
Step 4: Directed Fuzzing & Audit	Online	21,415	881.17	818.21 (92.9%)	19.67	900.84	10.16
Full Campaign Total (GCC + LLVM)	**All**	**59,671**	**1,545.59**	**1,427.42 (92.4%)**	**49.41**	**1,610.01**	**44.68**

[†]Step 1 ran on pre-sliced source and manual batches under a 15.00M-token cap (10.00M GCC, 5.00M LLVM); per-call cache counters are recorded across Steps 2–4.

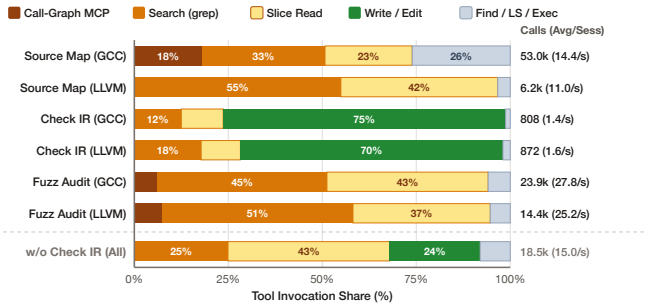


Fig. 5. Distribution and volume of agent MCP and workspace tool invocations across pipeline stages in DEFUZZ versus raw Go checker authoring (*w/o Check IR*).

Remaining Component Ablations. [TBD: Experiments for *w/o Hybrid RAG*, *w/o Source Mapping*, *w/o Programmatic Oracle*, and *Bare Agent* are reserved for completion. This paragraph will report invariant recall without RAG transfer, branch coverage without `DEFUZZ_HIT` probes, false positives under LLM adjudication, and stalls in unconstrained agents.]

E. RQ4: Agent Telemetry, Token Usage, and Overhead

Agent Tool-Call Telemetry. Figure 5 breaks down 99,236 tool calls across 6,798 agent sessions in DEFUZZ, plus 18,495 calls across 1,229 sessions in *w/o Check IR*. Tool patterns mirror the structure of each compiler. In GCC source mapping, agents issue 52,953 calls across 3,677 sessions (14.4 calls/session). Absent in LLVM, 18.0% of GCC calls query the Call-Graph MCP server to link flags across GIMPLE passes, RTL expanders, target hooks, and `.md` files. In LLVM, modular C++ pass files let agents map all 446 invariants in 6,244 calls (11.0 calls/session), using lexical search (54.8%) and slice reads (41.7%). In Step 3, Check IR authoring requires only **1,680 tool calls** across both compilers (1.4–1.6 calls/session), with over 70% directly writing concise specification files. Without Check IR, raw Go authoring requires **18,495 tool calls** (15.0 calls/session, **11.0× higher**) to search and inspect parser APIs. During online fuzzing and auditing, agents average 25.2–27.8 read-only navigation calls per session alongside 705 deterministic gRPC oracle runs.

Token Efficiency, Build Overhead, and Runtime. Table V reports LLM token usage and wall-clock time across all stages. One-time offline preparation (Steps 1–3) takes 709.17M tokens and 34.52 h, led by GCC source mapping (28.23 h versus 1.27 h on LLVM). Both the mapped branch hierarchies and compiled Check IR checkers are reusable across fuzzing runs and point releases. Deterministic AST instrumentation (Step 2b) rewrites 747 files with zero LLM calls. Building the instrumented compilers on 16 ARM Neoverse-N2 vCPUs takes 4.6 min (277.1 s) for `GCC_INST` and 25.8 min (1,550.8 s) for `CLANG_INST`. Because `DEFUZZ_HIT` probes stay dormant unless `DEFUZZ_LOG` is set, compiled target binaries incur zero runtime overhead. Online directed fuzzing (Step 4) completes in **10.16 h** (7.43 h GCC, 2.73 h LLVM) and consumes 900.84M tokens. Step 4 reaches its first verified silent defense bug after 31.9 min on GCC and 157.6 min on LLVM. Keeping invariant schemas, Ω signatures, and $\mathcal{T}_i$ fixed in prompt prefixes achieves a **92.4% overall KV-cache hit rate** (1,427.42M cached input tokens). As a result, online fuzzing costs an average of **4.8 min of wall-clock time and 7.15M tokens per discovered silent defense bug**.

REFERENCES

[1] MITRE, "CVE-2023-4039: GCC AArch64 stack protector bypass with variable-length arrays," https://nvd.nist.gov/vuln/detail/CVE-2023-4039, 2023.

[2] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2011.

[3] V. Livinskii, D. Babokin, and J. Regehr, "Random testing for C and C++ compilers with YARPGen," in *Proceedings of the ACM on Programming Languages (OOPSLA)*, 2020.

[4] GCC Bugzilla, "Bug 125079: tree-ssa-strlen get_string_length: __strcat_chk to __stpcpy_chk drops bound subtraction," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=125079, 2026.

[5] V. Le, M. Afshari, and Z. Su, "Compiler validation via equivalence modulo inputs," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2014.

[6] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[8] Y. Liu, Y. Xue, D. Wu, Y. Sun, Y. Li, M. Shi, and Y. Liu, "PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2025.
[9] GCC Bugzilla, "Bug 96191: aarch64 stack_protect_test canary leak," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=96191, 2020.
[10] J. Regehr *et al.*, "C-Reduce: A C and C++ program reducer," https://github.com/csmith-project/creduce, 2012.
[11] GCC Bugzilla, "Bug 128276: LTO stream-in drops HWASan/MTE stack scope markers," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=128276, 2026.
[12] ——, "Bug 128011: RISC-V Zicfilp emits patchable-function-entry NOPs before landing pad," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=128011, 2026.
[13] ——, "Bug 127318: RISC-V RVV stack-clash protection switches CFA register to t3 prematurely," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=127318, 2026.
[14] ——, "Bug 125049: LoongArch stack_protect_test leaves canary value in caller-visible register," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=125049, 2026.
[15] LLVM GitHub Issues, "Issue 236673: [AArch64][KCFI] LowerKCFI_CHECK clobbers live argument register w9 under preserve_none," https://github.com/llvm/llvm-project/issues/236673, 2026.
[16] ——, "Issue 237862: [AArch64][PAC] Function pointer in data-key __ptrauth slot authenticated with IA key," https://github.com/llvm/llvm-project/issues/237862, 2026.
[17] GCC Bugzilla, "Bug 128271: [x86] CET shadow-stack clrssbsy/wrss* patterns omit FLAGS_REG and memory clobbers," https://gcc.gnu.org/bugzilla/show_bug.cgi?id=128271, 2026.
[18] LLVM GitHub Issues, "Issue 237864: [SafeStack][AArch64] Scalable SVE local moved to unsafe stack gets fixed 16-byte slot," https://github.com/llvm/llvm-project/issues/237864, 2026.